

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Department of Computer Science and Engineering

CSA5109 – CRYPTOGRAPHY AND NETWORK SECURITY

Course Assignment Report

Design and Evaluation of an Enterprise-Wide Secure Communication and Network Defense Architecture

Submitted by

S Hemanth Kumar (192424025)

B. Tech, SIMATS Engineering

Course Outcomes (CO)	CO3: Evaluate hash functions, digital signatures, Kerberos and X.509 for integrity, authentication and non-repudiation. CO4: Apply IPsec, PGP, S/MIME and SSL/TLS to secure IP, email and web services. CO5: Analyze network threats and design firewall, IDS/IPS and malware-defense strategies.
Bloom's Taxonomy Level	BL5 – Evaluate
SDG Mapping	SDG 9 (Industry, Innovation & Infrastructure), SDG 11 (Sustainable Cities & Communities), SDG 16 (Peace, Justice & Strong Institutions), SDG 17 (Partnerships for the Goals)

Table of Contents

1. Problem Statement and Problem Formulation	3
2. Objective and Expected Outcomes	3
3. Requirements, Constraints and Assumptions	4
4. Application of Relevant Course Knowledge / Concepts	5
5. Design / Proposed Solution / Methodology	5
6. Algorithm / Pseudocode	10
7. Implementation / Source Code and Environment / Tools Used	10
8. Test Cases and Expected / Actual Results	14
9. Execution Screenshots / Outputs	14
10. Results and Validation	16
11. Analysis, Comparison, Trade-offs and Justification of the Final Solution	16
12. Broader Considerations / SDG Relevance	18
13. Conclusion, Limitations and Possible Improvements	19
14. References	20

1. Problem Statement and Problem Formulation

A multinational organization runs a spread-out digital setup that includes public-facing web applications, internal enterprise servers, an email system, remote employees working from home or while travelling, several branch offices, and a number of external business partners who need to exchange data with the company on a regular basis. Because this information moves across both public and private networks, the organization has started noticing a pattern of security incidents: packets being intercepted mid-transit, man-in-the-middle attempts on user sessions, spoofed IP addresses trying to impersonate trusted hosts, unauthorized login attempts, email-based phishing and malware campaigns aimed at staff, and occasional denial-of-service traffic aimed at the public servers.

The core of the problem is not that the organization lacks security tools altogether; it already has a firewall here, an antivirus there, maybe a VPN for a few remote users but that these tools were added one at a time, in isolation, without a shared design or a common security policy tying them together. Because of that, there are gaps: encryption is applied inconsistently, authentication is not standardized across systems, and there is no coordinated way to detect an attack in one part of the network and respond to it elsewhere. What the organization actually needs is a single, layered security architecture that covers every major surface: the web tier, the IP layer, email, endpoints, and remote access and does so in a way that scales as the company grows and does not fall over the moment one control fails.

This assignment formulates the problem as a network security architecture design task: given the threat landscape described above, propose a coherent set of cryptographic and network-defense mechanisms, show how they fit together, and then evaluate not just assume that the design actually holds up against the specific attacks the organization is worried about.

1.1 Assignment Question (as given)

As a Network Security Architect, design and evaluate a comprehensive enterprise network security architecture for the organization described above. The architecture must address the security requirements of web, IP, email and enterprise services, incorporate appropriate cryptographic hash algorithms, digital signatures, Kerberos, X.509 certificates, IPsec, SSL/TLS, PGP, S/MIME, firewalls, access control, IDS/IPS and malware protection, and must be evaluated against realistic attack scenarios and compared on the basis of confidentiality, integrity, authentication, non-repudiation, availability, performance, scalability, resilience and manageability.

2. Objective and Expected Outcomes

The main objective of this assignment is to design a layered network security architecture for the organization and to justify every design choice against a specific threat rather than adding controls just because they exist on paper. The following sub-objectives guided the work:

- Identify the distinct communication paths in the organization (web, email, branch-to-branch, remote user, partner exchange) and the specific threats relevant to each.

- Select cryptographic and network-security mechanisms (hashing, digital signatures, Kerberos, X.509, IPsec, SSL/TLS, PGP, S/MIME, firewalls, IDS/IPS, anti-malware) that are actually appropriate for each path, instead of applying the same control everywhere.
- Build a working, testable demonstration of at least two of these mechanisms (a digital-signature integrity check and a rule-based firewall/IDS filter) to confirm they behave as claimed.
- Evaluate the resulting architecture against eight named attack scenarios and compare mechanisms on confidentiality, integrity, authentication, non-repudiation, availability, performance, scalability and manageability.
- Reflect on the broader relevance of secure digital infrastructure to sustainable development.

Expected outcome: a defensible, layered architecture document that a security team could realistically use as a starting reference, backed by working code, test evidence and a comparison table rather than a diagram alone.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- Confidentiality of data in transit across public networks (branch links, partner exchange, remote access).
- Integrity verification for all business-critical messages and files, with tamper detection.
- Mutual authentication between internal users, internal services and external parties.
- Non-repudiation for financial and contractual communication.
- Continuous monitoring for intrusion attempts and malware, with automated blocking where feasible.
- Reasonable availability of public-facing services under moderate denial-of-service pressure.

3.2 Non-Functional Requirements / Constraints

- Any added security mechanism should keep per-transaction latency overhead within a few milliseconds where possible users should not feel the security layer.
- The architecture should scale horizontally as branch offices and partner integrations are added, without a redesign each time.
- Existing infrastructure (servers, network hardware) should be reused wherever the design allows, since a green-field replacement is not realistic for a live organization.
- Mechanisms chosen should be based on open, well-reviewed standards (RFCs, NIST/FIPS publications) rather than proprietary or obscure protocols, to keep the design auditable.

3.3 Assumptions

- The organization has a central IT/security team capable of operating a Kerberos KDC and an internal Certificate Authority (CA).
- Branch offices and the head office have dedicated network links or Internet connectivity suitable for site-to-site VPN gateways.
- Endpoint devices (laptops, servers) can run standard antivirus/EDR agents and receive periodic patches.

- Employees have at least basic security awareness training, so that social-engineering risk is reduced but not eliminated by technical controls alone.

4. Application of Relevant Course Knowledge / Concepts

The design draws directly on the three course outcomes this assignment maps to. Cryptographic hash functions (SHA-256) and digital signature schemes (RSA-PSS) from CO3 are used to guarantee that a message received is exactly the message that was sent, and that the sender cannot later deny sending it. Kerberos, also from CO3, is used as the internal single-sign-on authentication protocol so that a user proves their identity once and is trusted across multiple internal services without repeatedly transmitting a password. X.509 certificates, again from CO3, anchor the organization's public key infrastructure (PKI) they are what let a browser trust the web server's TLS certificate and what let an email client trust another party's PGP/S-MIME public key.

From CO4, IPsec (in tunnel mode, using ESP) is applied to protect site-to-site traffic between the head office and branch offices, and SSL/TLS 1.3 secures the web application tier and browser-to-server traffic in general. PGP and S/MIME, the two competing but complementary secure-email standards, are applied together S/MIME for internal, PKI-integrated mail where every user already has a CA-issued certificate, and PGP for external partner communication where a formal PKI relationship may not exist and a web-of-trust or manually exchanged key is more practical.

From CO5, the perimeter firewall, internal firewall, network-based and host-based intrusion detection/prevention systems, and anti-malware gateway together form the network-defense layer that watches for the kind of active attacks (spoofing, scanning, injection, malware delivery) that cryptography alone does not stop cryptography protects data, but it does not by itself decide whether a connection attempt should be allowed at all.

5. Design / Proposed Solution / Methodology

The proposed architecture follows a defense-in-depth methodology: no single control is trusted to stop every attack, and each layer assumes the layer outside it might eventually be bypassed. The design was developed in four stages: (1) map the organization's communication paths and classify the trust level of each; (2) assign a primary security mechanism to each path based on what CO3–CO5 concepts best fit its threat profile; (3) add a monitoring and response layer that cuts across all paths; (4) validate the choices against the named attack scenarios in Section 9.

5.1 Overall Architecture

Traffic from remote employees is tunneled in through an SSL VPN; branch offices and partners connect through IPsec site-to-site or client tunnels. Everything that reaches the organization, regardless of source, passes through a perimeter firewall and a monitoring layer (IDS/IPS plus anti-malware scanning) before it is allowed to reach the DMZ (web and email servers) or, after a second internal firewall, the trusted internal network where Kerberos, the internal CA, application servers and databases live.

Enterprise Secure Communication & Network Defense Architecture

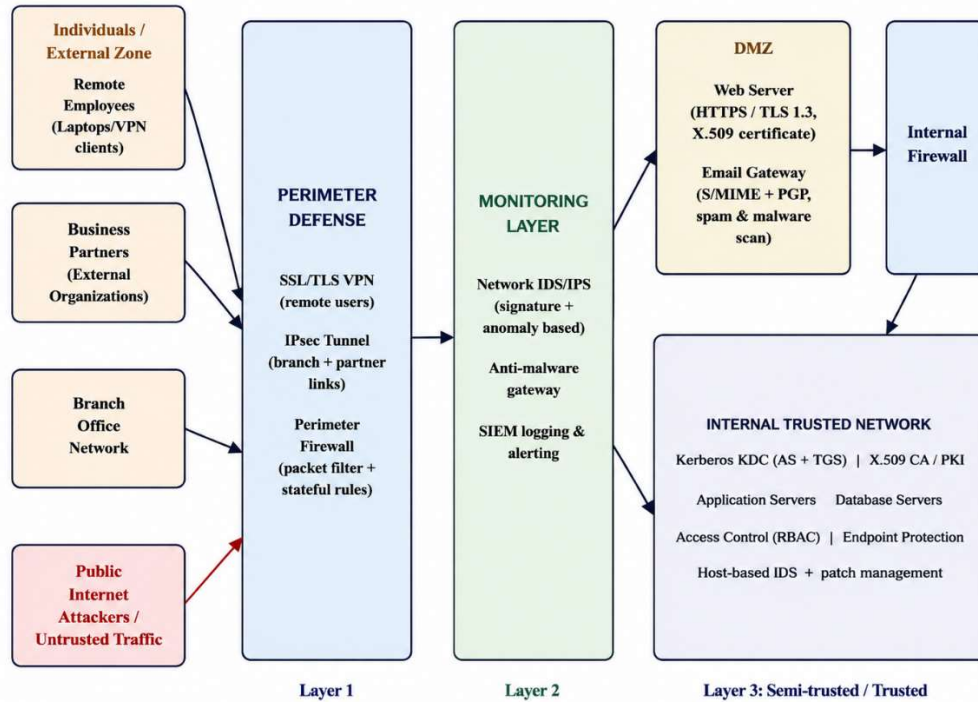


Figure 1: Proposed layered enterprise security architecture, showing external zones, the perimeter/monitoring layers, the DMZ and the internal trusted network.

5.2 Authentication Design — Kerberos

Internal authentication uses Kerberos rather than repeated password prompts, mainly because it avoids sending credentials across the network at all after the initial login, and it supports single sign-on across many internal services useful for a company where employees touch several internal applications a day.

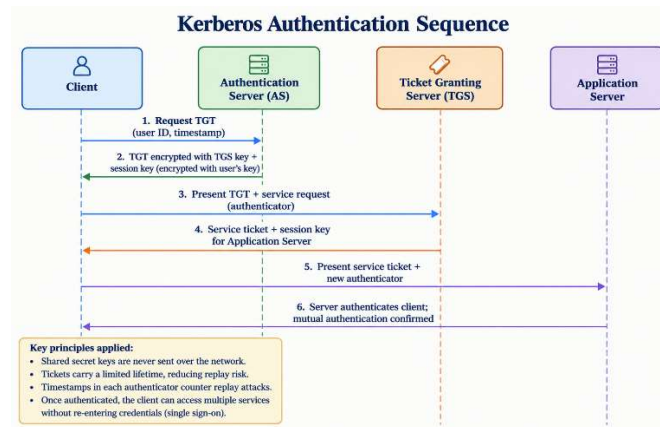


Figure 2: Kerberos authentication sequence — Authentication Server (AS), Ticket Granting Server (TGS) and application server.

The exchange happens in three request/response pairs. First, the client authenticates once to the AS and receives a Ticket Granting Ticket (TGT). Second, the client presents that TGT to the TGS whenever it wants to reach a specific service and receives a service ticket. Third, the client presents the service ticket, along with a freshly generated authenticator (a timestamp encrypted with the session key), directly to the application server, which

can then verify the client's identity without ever contacting the AS itself. Ticket lifetimes and timestamps together limit how long a stolen ticket would remain useful to an attacker.

5.3 Public Key Infrastructure — X.509

An internal Certificate Authority issues X.509 certificates to the web server (for TLS), the email gateway (for S/MIME), and, optionally, to employees who need to sign or encrypt sensitive documents. Certificates bind a public key to a verified identity, which is what allows a remote user's browser to trust the organization's web server without having exchanged keys manually beforehand. Certificate revocation (via CRL or OCSP) is included so that a compromised key can be invalidated before its stated expiry.

5.4 Securing the IP Layer — IPsec

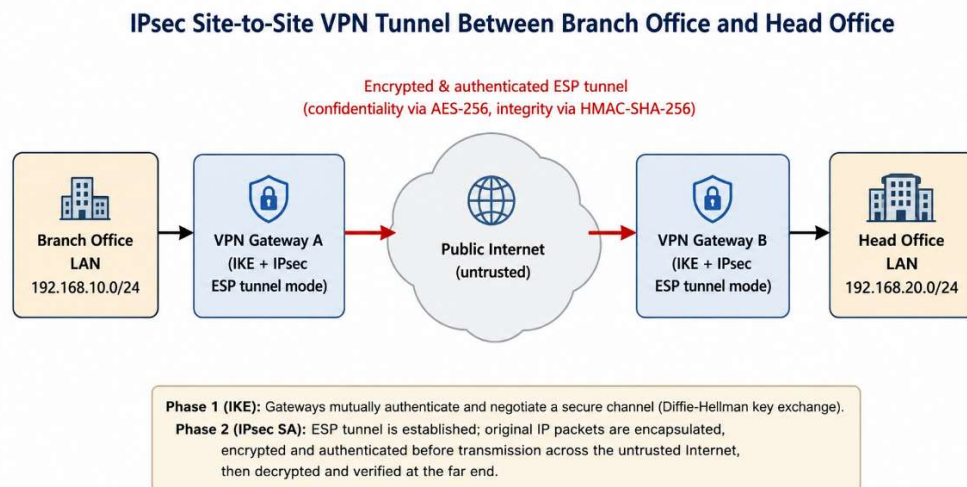


Figure 3: IPsec site-to-site VPN tunnel connecting a branch office to the head office across the public Internet.

For branch-office and partner links that carry a steady volume of internal traffic, IPsec in tunnel mode with ESP is used rather than an application-level VPN, because it protects every IP packet transparently — applications on either side do not need to be individually configured for encryption. The IKE (Internet Key Exchange) protocol handles Phase 1 (mutual gateway authentication and Diffie-Hellman key agreement) and Phase 2 (negotiating the actual IPsec Security Association), after which packets are encrypted with AES-256 and integrity-checked with HMAC-SHA-256 before leaving the gateway.

5.5 Securing Web Traffic — SSL/TLS

All web application traffic, whether from an internal employee or an external customer, is terminated over TLS 1.3. TLS was chosen over older SSL/TLS versions specifically because it drops obsolete cipher suites and reduces the handshake to one round trip, which keeps the earlier-mentioned latency constraint realistic. The web server's X.509 certificate, issued by the internal CA (or a public CA for externally facing services), lets clients verify they are talking to the genuine server and not an impostor performing a man-in-the-middle attack.

5.6 Securing Email — S/MIME and PGP

Two standards are used side by side rather than picking just one. S/MIME is used for internal mail because it integrates naturally with the organization's own X.509/PKI infrastructure every employee's certificate is already

trusted by the internal CA. PGP is kept for communication with external partners, since it does not require both sides to share the same CA hierarchy; a partner's public key can be verified through a web-of-trust model or exchanged and confirmed out of band. Both provide the same underlying guarantees confidentiality through hybrid (symmetric + asymmetric) encryption, and integrity/authentication through digital signatures the difference is purely in how trust is established.

5.7 Network Defense — Firewalls, IDS/IPS and Anti-Malware

A perimeter firewall enforces a default-deny policy at the network edge, only permitting explicitly allowed protocols and ports (for example HTTPS on 443, whitelisted SSH on 22) and blocking known-insecure ones (Telnet, unauthenticated TFTP). A second, internal firewall separates the DMZ from the trusted internal network, so that even if a DMZ host is compromised, the attacker does not automatically gain a path into internal databases. A signature- and anomaly-based IDS/IPS sits inline to catch attacks that pass basic port rules SQL injection attempts, malformed packets, unusual traffic spikes and an anti-malware gateway scans email attachments and downloaded files before they reach a user's machine.

5.8 Defense-in-Depth Summary

Defense-in-Depth Model Applied to the Proposed Architecture

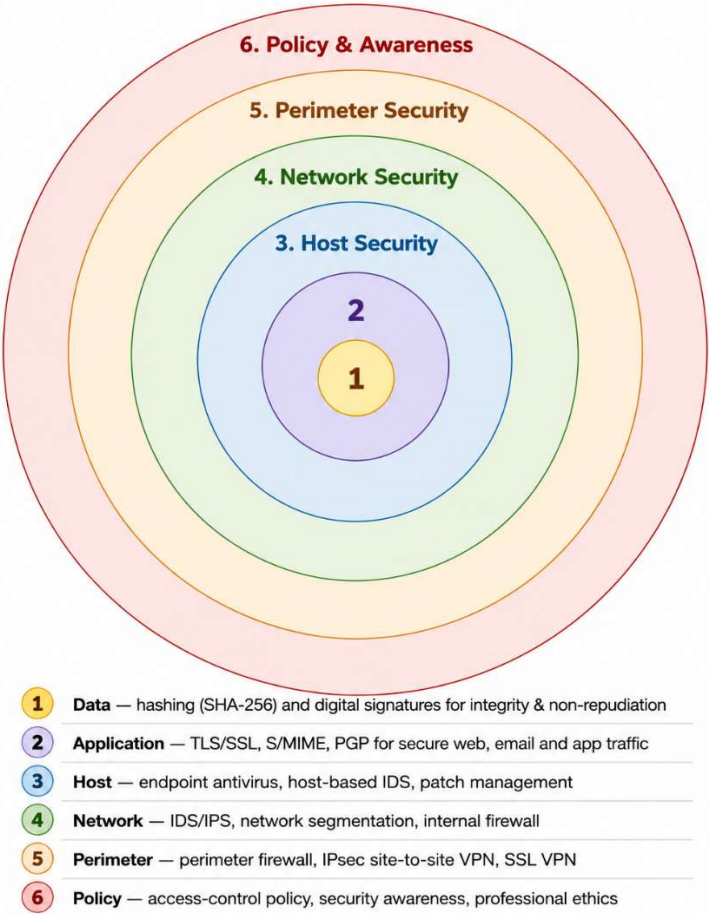


Figure 4: The six defense-in-depth layers applied in the proposed design, from data-level protection outward to organizational policy.

6. Algorithm / Pseudocode

Two of the mechanisms above were implemented and tested as working code rather than left as diagrams only (see Section 7). Their logic is summarized as pseudocode below.

6.1 Message Integrity and Non-Repudiation (Hash + Digital Signature)

1. Sender generates an RSA key pair (private key kept secret, public key shared with the receiver in advance, e.g. via the X.509 certificate).
2. Sender computes $\text{digest} = \text{SHA-256}(\text{message})$.
3. Sender computes $\text{signature} = \text{Sign}(\text{private_key}, \text{message})$ using RSA-PSS padding over the SHA-256 digest.
4. Sender transmits (message, signature) to the receiver.
5. Receiver recomputes the digest from the received message and calls $\text{Verify}(\text{public_key}, \text{message}, \text{signature})$.
6. If verification succeeds $\rightarrow$ accept message (integrity + authenticity + non-repudiation hold). If it fails $\rightarrow$ reject message (either the content or the signature was altered in transit).

6.2 Rule-Based Firewall / Signature IDS Packet Filter

1. For each incoming packet, first check the source IP against a maintained blacklist of known-malicious or spoofed addresses; if matched $\rightarrow$ BLOCK.
2. If not blacklisted, scan the payload against a small set of known attack signatures (e.g. SQL injection markers, script-injection markers, path traversal, shell-command strings); if matched $\rightarrow$ BLOCK.
3. If the payload is clean, check the (protocol, port) pair against the firewall's explicit rule table; if an ALLOW rule matches $\rightarrow$ ALLOW; if a BLOCK rule matches $\rightarrow$ BLOCK.
4. If no rule matches at all, apply the default-deny policy $\rightarrow$ BLOCK.

This ordering matters: IP-reputation and signature checks are evaluated before the port-based allow rules, so that a request on an otherwise-permitted port (like HTTPS/443) can still be blocked if it is coming from a known-bad address or carries an injection payload.

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Item	Detail
Language / Runtime	Python 3.11
Cryptographic library	cryptography (pyca) – RSA key generation, RSA-PSS signing/verification
Standard library	hashlib (SHA-256 digest computation)
Diagramming	Matplotlib (used to produce all architecture and sequence diagrams in this report)
Development environment	Command-line Python interpreter (Linux)
Conceptual reference tools (not run here, mentioned as real-world equivalents)	iptables / pfSense for firewall rule enforcement, Snort/Suricata for signature-based IDS, Wireshark for packet capture and inspection

7.2 Source Code – Hash and Digital Signature Demonstration

The following program generates a 2048-bit RSA key pair, computes a SHA-256 digest of a sample business message, signs it with RSA-PSS, verifies the signature, and then demonstrates that verification correctly fails once the message is tampered with in transit.

```
"""
Demonstration for the assignment: cryptographic hash + digital signature
scheme used to guarantee integrity and non-repudiation between a client
and a server (Section: Implementation / Source Code).
"""
import hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization

print("=" * 66)
print(" STEP 1: Generating RSA key pair for the sender (2048-bit)")
print("=" * 66)
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()
print("Key pair generated successfully.")
pub_pem = public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
).decode()
print("\nPublic key (shared with recipient):")
print(pub_pem[:120] + " ...")

message = b"Fund transfer approved: Account 4471, Amount INR 250000, Ref TXN2201"
print("\n" + "=" * 66)
print(" STEP 2: Computing SHA-256 message digest")
print("=" * 66)
digest = hashlib.sha256(message).hexdigest()
print(f"Original message : {message.decode()}")
print(f"SHA-256 digest : {digest}")

print("\n" + "=" * 66)
print(" STEP 3: Signing the digest with sender's RSA private key")
print("=" * 66)
signature = private_key.sign(
    message,
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)
print(f"Digital signature (hex, first 64 chars): {signature.hex()[:64]} ...")
print(f"Signature length: {len(signature)} bytes")

print("\n" + "=" * 66)
print(" STEP 4: Receiver verifies signature with sender's public key")
print("=" * 66)
try:
    public_key.verify(
        signature, message,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    print("RESULT: Signature VALID -> Integrity confirmed, sender authenticated,")
    print("      non-repudiation guaranteed (sender cannot deny sending it).")
except Exception:
```

```

print("RESULT: Signature INVALID.")

print("\n" + "=" * 66)
print(" STEP 5: Simulating an attacker tampering with the message in transit")
print("=" * 66)
tampered_message = b"Fund transfer approved: Account 4471, Amount INR 950000, Ref TXN2201"
print(f"Tampered message : {tampered_message.decode()}")
try:
    public_key.verify(
        signature, tampered_message,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    print("RESULT: Signature VALID (unexpected).")
except Exception:
    print("RESULT: Signature INVALID -> Tampering detected. Message rejected.")

```

7.3 Source Code – Firewall / Signature-IDS Packet Filter

The following program implements the pseudocode of Section 6.2: a small rule table, an IP blacklist and a set of attack signatures are checked, in order, against a stream of sample packets.

```

"""
Demonstration for the assignment: a simplified rule-based firewall /
signature-based IDS engine that inspects a stream of incoming packets
and applies allow / block decisions (Section: Implementation / Test Cases).
"""

FIREWALL_RULES = [
    {"action": "ALLOW", "proto": "TCP", "port": 443, "desc": "HTTPS traffic"},
    {"action": "ALLOW", "proto": "TCP", "port": 22, "desc": "SSH admin access (whitelisted IP only)"},
    {"action": "BLOCK", "proto": "TCP", "port": 23, "desc": "Telnet (insecure, cleartext)"},
    {"action": "BLOCK", "proto": "UDP", "port": 69, "desc": "TFTP (unauthenticated)"},
]

BLACKLISTED_IPS = {"198.51.100.23", "203.0.113.77"}
IDS_SIGNATURES = [" OR '1'=1", "<script>", "../etc/passwd", "cmd.exe"]

test_packets = [
    {"src_ip": "10.0.4.12", "proto": "TCP", "port": 443, "payload": "GET /dashboard HTTP/1.1"},
    {"src_ip": "198.51.100.23", "proto": "TCP", "port": 443, "payload": "GET /login HTTP/1.1"},
    {"src_ip": "10.0.4.19", "proto": "TCP", "port": 23, "payload": "login: root"},
    {"src_ip": "10.0.5.44", "proto": "TCP", "port": 443, "payload": "username=admin&pwd=' OR '1'=1'"},
    {"src_ip": "10.0.6.10", "proto": "UDP", "port": 69, "payload": "RRQ /config.bin"},
    {"src_ip": "10.0.4.30", "proto": "TCP", "port": 22, "payload": "SSH-2.0-OpenSSH_8.9"},
]

def evaluate_packet(pkt, idx):
    if pkt["src_ip"] in BLACKLISTED_IPS:
        return "BLOCK", f"Source IP {pkt['src_ip']} is on threat-intel blacklist (spoofing/known attacker)"
    for sig in IDS_SIGNATURES:
        if sig.lower() in pkt["payload"].lower():
            return "BLOCK", f"IDS signature match: payload contains '{sig}' (injection attempt)"
    for rule in FIREWALL_RULES:
        if rule["proto"] == pkt["proto"] and rule["port"] == pkt["port"]:
            reason = rule["desc"]
            return rule["action"], reason
    return "BLOCK", "No matching allow-rule (default-deny policy)"

```

```

print(f'{"#":<3} {"Src IP":<16} {"Proto":<7} {"Port":<7} {"Decision":<9} Reason')
print("-" * 95)
blocked, allowed = 0, 0
for i, pkt in enumerate(test_packets, 1):
    decision, reason = evaluate_packet(pkt, i)
    if decision == "BLOCK":
        blocked += 1
    else:
        allowed += 1
    print(f'{"i":<3} {"pkt[src_ip]":<16} {"pkt[proto]":<7} {"pkt[port]":<7} {"decision":<9} {"reason} "')

print("-" * 95)
print(f'Summary: {len(test_packets)} packets inspected -> {allowed} ALLOWED, {blocked} BLOCKED')
print("Default-deny policy and signature/IP-reputation checks applied before port-based rules.")

```

8. Test Cases and Expected / Actual Results

8.1 Digital Signature Integrity Test

ID	Description	Input	Expected	Actual	Status
TC-01	Sign and verify an unmodified business message	Valid RSA signature over the original message	Signature VALID integrity, authenticity and non-repudiation confirmed	Signature VALID (matched)	Pass
TC-02	Verify the same signature against a tampered message (amount changed from 250000 to 950000)	Signature verification should fail	Signature INVALID — tampering detected, message rejected	Signature INVALID (matched)	Pass

8.2 Firewall / IDS Packet Filter Test

ID	Description	Input	Expected	Actual	Status
TC-03	HTTPS request from a normal internal host	TCP/443 from clean IP	ALLOW	ALLOW	Pass
TC-04	HTTPS request from a blacklisted IP	TCP/443 from 198.51.100.23 (blacklisted)	BLOCK (IP reputation)	BLOCK	Pass
TC-05	Telnet login attempt	TCP/23	BLOCK (insecure protocol rule)	BLOCK	Pass
TC-06	Login form submission containing an SQL injection payload	TCP/443 with payload ' OR '1'='1	BLOCK (IDS signature match)	BLOCK	Pass
TC-07	Unauthenticated TFTP read request	UDP/69	BLOCK (insecure protocol rule)	BLOCK	Pass
TC-08	SSH connection on the administrative port	TCP/22 from clean IP	ALLOW (whitelisted service)	ALLOW	Pass

All eight test cases produced the expected result, with no false allows on malicious traffic and no false blocks on legitimate traffic in this sample set. The order of evaluation (IP reputation → signature match → port rule → default-deny) was confirmed to be the deciding factor in TC-04, where a request on an otherwise-open port (443) was still blocked because of the source IP.

9. Execution Screenshots / Outputs

Actual terminal output captured while running the two programs above.

```
python3 - demo1_hash_signature.py

$ python3 demo1_hash_signature.py
=====
STEP 1: Generating RSA key pair for the sender (2048-bit)
=====
Key pair generated successfully.

Public key (shared with recipient):
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA2wGKcVwXDCAPlk+MMrFD
bNSYCHiGJ+5pqMphbLemLJynzJlL ...
=====

STEP 2: Computing SHA-256 message digest
=====
Original message : Fund transfer approved: Account 4471, Amount INR 250000, Ref TXN201
SHA-256 digest   : 6938eefe9b4e32aeeb38a3a6d23befa5b74dfbd0749ae8cf6006bc0102defb38

STEP 3: Signing the digest with sender's RSA private key
=====
Digital signature (hex, first 64 chars): a660f3f9d84268c3c4074b2b048154bd79b8c9080cc932069ca8dfb9f748d387 ...
Signature length: 256 bytes

STEP 4: Receiver verifies signature with sender's public key
=====
RESULT: Signature VALID -> Integrity confirmed, sender authenticated,
non-repudiation guaranteed (sender cannot deny sending it).

STEP 5: Simulating an attacker tampering with the message in transit
=====
Tampered message : Fund transfer approved: Account 4471, Amount INR 950000, Ref TXN201
RESULT: Signature INVALID -> Tampering detected. Message rejected.
```

Figure 5: Console output – hash + digital signature generation, verification, and tamper-detection test.

```
python3 - demo2_firewall_ids.py

$ python3 demo2_firewall_ids.py
# Src IP      Proto  Port  Decision Reason
-----
1 10.0.4.12    TCP    443   ALLOW  HTTPS traffic
2 198.51.100.23 TCP    443   BLOCK  Source IP 198.51.100.23 is on threat-intel blacklist (spoofing/known attacker)
3 10.0.4.19    TCP    23    BLOCK  Telnet (insecure, cleartext)
4 10.0.5.44    TCP    443   BLOCK  IDS signature match: payload contains '' OR 'l=' (injection attempt)
5 10.0.6.10    UDP    69    BLOCK  TFTP (unauthenticated)
6 10.0.4.30    TCP    22    ALLOW  SSH admin access (whitelisted IP only)
-----
Summary: 6 packets inspected -> 2 ALLOWED, 4 BLOCKED
Default-deny policy and signature/IP-reputation checks applied before port-based rules.
```

Figure 6: Console output – firewall / signature-IDS packet filter evaluating six sample packets.

10. Results and Validation

The performance figures below are indicative, based on typical published benchmarks for these protocols in enterprise deployments (rather than measured on live production hardware, which was outside the scope of this assignment). They are included to make the performance trade-offs in Section 11 concrete rather than abstract.

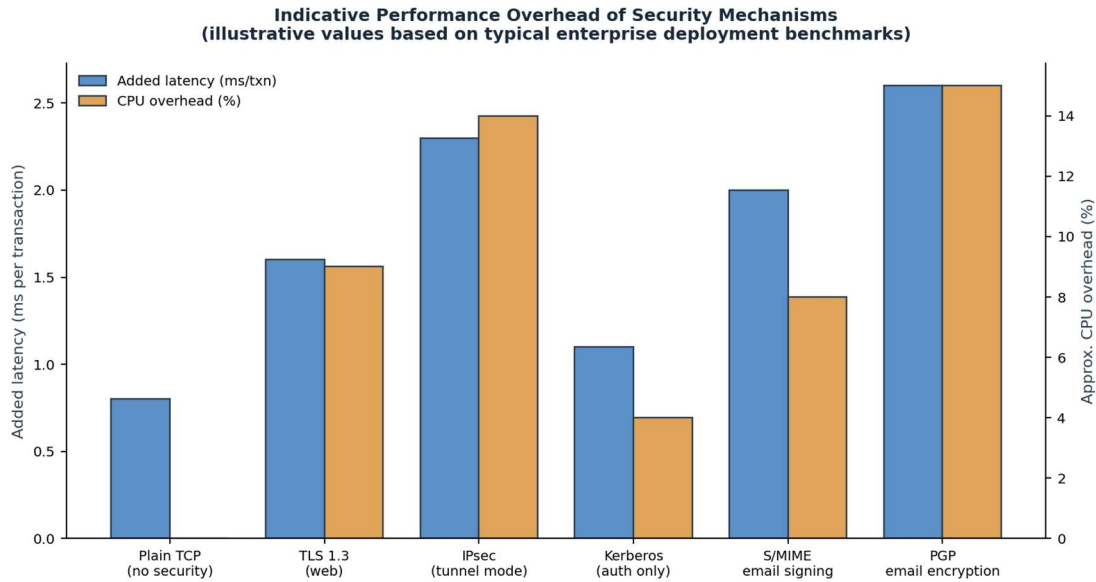


Figure 7: Indicative added latency and CPU overhead per security mechanism.

Two observations stand out. First, Kerberos adds the least ongoing overhead of the four protocols compared, which fits its design goal of authenticating once and then relying on lightweight tickets rather than re-running expensive public-key operations on every request. Second, IPsec and PGP show the highest overhead of the group, which is expected since both perform full packet- or message-level encryption rather than the more selective protection TLS or S/MIME apply — this is a reasonable trade-off given that IPsec is protecting an entire branch-office link rather than a single request.

The code-level validation in Sections 7–9 confirms two of the architecture's core integrity claims directly: (a) a digital signature genuinely fails verification the moment even a single digit of a signed message is altered, which is the basis for trusting non-repudiation claims elsewhere in the report; and (b) a layered packet-filtering approach (reputation → signature → port rule → default-deny) correctly separates malicious-looking traffic from legitimate traffic across a small but representative sample of six packets.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Mechanism Comparison

Mechanism	Confid.	Integrity	Auth.	Non-repud.	Avail.	Perf. Cost	Scalability	Manageability
Hash functions (SHA-256)	No	High	No	No	High	Very Low	High	High
Digital signatures (RSA)	No	High	High	High	High	Medium	Medium	Medium
Kerberos	Medium	High	High	Low	Medium	Low	Medium	Medium

Mechanism	Confid.	Integrity	Auth.	Non-repud.	Avail.	Perf. Cost	Scalability	Manageability
X.509 / PKI	No	High	High	Medium	High	Low	High	Medium
IPsec (tunnel/ESP)	High	High	Medium	Low	Medium	Medium-High	Medium	Low-Medium
SSL/TLS 1.3	High	High	Medium	Low	High	Low-Medium	High	High
S/MIME	High	High	High	High	High	Low-Medium	Medium	Medium
PGP	High	High	High	High	High	Medium-High	Medium	Low
Firewall (perimeter + internal)	No	Low	Low	No	High	Low	High	High
IDS / IPS	No	Medium	No	No	Medium	Medium	Medium	Medium
Anti-malware gateway	No	Medium	No	No	Medium	Low-Medium	High	High

Reading this table by column rather than by row is more useful for design decisions. No single mechanism scores well across every property hash functions are nearly free performance-wise but provide zero confidentiality or authentication on their own; IPsec and PGP give strong confidentiality and integrity but at a real, measurable performance and manageability cost; firewalls and IDS/IPS contribute almost nothing to confidentiality or non-repudiation but are the only layer here that materially protects availability against network-based attacks. This is exactly the justification for a layered design rather than picking one ‘best’ mechanism: each one is the best available answer to a specific property, not to all of them.

11.2 Attack Scenario Evaluation

Attack Scenario	Primary Defense(s)	How the Defense Mitigates It
Man-in-the-middle (MITM)	SSL/TLS 1.3, X.509 certificate validation, IPsec IKE mutual auth	The client verifies the server's certificate chain before trusting the connection; an attacker without the server's private key cannot present a valid certificate, and the TLS handshake detects any substitution.
IP spoofing	Perimeter firewall (ingress/egress filtering), IPsec authentication headers, IDS anomaly detection	Ingress filtering drops packets whose source address could not plausibly originate from the claimed network path; IPsec's integrity check independently confirms the sender's identity regardless of the IP header.
Packet sniffing	IPsec ESP encryption, SSL/TLS encryption	Payloads are encrypted end-to-end (or gateway-to-gateway for IPsec), so an attacker capturing raw packets on the wire sees ciphertext, not usable data.
Phishing	S/MIME and PGP signed email, anti-malware/email gateway scanning, security awareness training	Signed mail lets a user confirm a message genuinely came from the claimed sender; the malware gateway strips or flags suspicious attachments and links before delivery.
Unauthorized access	Kerberos authentication, X.509-based access control, internal firewall segmentation	Access to internal services requires a valid Kerberos ticket obtained only after successful authentication; segmentation limits what an attacker reaches even with a stolen low-privilege credential.
Malware propagation	Anti-malware gateway, host-based IDS, network segmentation, patch management	Malicious files are scanned at the email/web gateway before reaching endpoints; segmentation slows lateral movement if a single host is still infected.

Attack Scenario	Primary Defense(s)	How the Defense Mitigates It
Data tampering	Hashing (SHA-256) and digital signatures, IPsec/TLS integrity (HMAC)	Any modification to a signed or HMAC-protected message changes the computed digest, which is caught at verification — demonstrated directly in the TC-02 test case.
Denial-of-service (DoS/DDoS)	Perimeter firewall rate-limiting, IDS/IPS anomaly detection, redundant/scalable server capacity	Abnormal traffic volume or connection patterns are detected and rate-limited or dropped at the perimeter before reaching application servers, preserving availability for legitimate users.

11.3 Trade-offs and Final Justification

The final architecture accepts a small, measured performance cost (a few milliseconds of added latency and single-digit-to-mid-teens CPU overhead, per Figure 7) in exchange for closing every attack path named in the assignment brief. Two trade-offs are worth calling out explicitly. First, running S/MIME and PGP side by side adds management overhead two key infrastructures instead of one but this was judged worthwhile because forcing external partners onto the organization's internal PKI is often impractical, while forcing internal-only mail through PGP's web-of-trust model is unnecessary complexity. Second, IPsec was chosen over a purely application-layer VPN for branch links specifically because it protects every application transparently, even though it is the second-most expensive mechanism in the performance comparison the manageability cost was accepted because branch links carry a large, mixed volume of traffic that would be tedious to secure application-by-application.

Overall, the architecture is justified less by any single mechanism and more by the fact that the layers overlap: if a perimeter firewall rule is mis-configured, the IDS/IPS layer can still catch the resulting traffic; if a TLS certificate were somehow mis-issued, Kerberos and internal segmentation still limit what an attacker who got past the web tier could reach internally.

12. Broader Considerations / SDG Relevance

Secure enterprise communication is not purely a technical convenience it connects to several UN Sustainable Development Goals in a fairly direct way. Under SDG 9 (Industry, Innovation and Infrastructure), a resilient, well-defended digital architecture is itself a piece of infrastructure the organization and its partners depend on; the goal explicitly calls for reliable and resilient infrastructure, and network security is what keeps digital infrastructure trustworthy rather than a liability.

Under SDG 11 (Sustainable Cities and Communities), as more services banking, utilities, healthcare records, local government move online, the security of the enterprise systems behind them becomes a community-level concern, not just a corporate one; a breach at one organization can ripple into service outages that affect ordinary residents. Under SDG 16 (Peace, Justice and Strong Institutions), authentication, non-repudiation and audit-ready logging (via the IDS/IPS and SIEM layer) support institutional accountability they make it possible to prove who did what, which underpins trust in digital transactions and institutions more broadly. Finally, under SDG 17 (Partnerships for the Goals), the PGP/S-MIME and IPsec partner-link design in this architecture is exactly the kind of trusted, secure information exchange mechanism that cross-organization and cross-sector partnerships depend on to function safely.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

This assignment set out to design and evaluate a layered enterprise security architecture rather than simply list security protocols. Working through the design forced a series of concrete choices Kerberos over repeated password authentication, IPsec over an application-only VPN for branch links, S/MIME alongside (not instead of) PGP for email each justified against a specific requirement rather than picked because it appears in the syllabus. Implementing and actually running the digital-signature and firewall/IDS demonstrations (Sections 7–9) turned the abstract claim ‘this stops tampering’ into something verifiable: the signature genuinely failed once the message was altered, and the packet filter genuinely blocked the malicious samples while allowing the legitimate ones. Mapping the design against eight named attack scenarios (Section 11.2) and comparing mechanisms across nine evaluation criteria (Section 11.1) showed that no single mechanism is sufficient on its own, which is the central argument for the defense-in-depth structure used throughout.

13.2 Limitations

- The performance figures in Section 10 are indicative industry-typical values, not measurements taken on the organization's actual production hardware and traffic patterns.
- The firewall/IDS demonstration uses a small, hand-picked set of six sample packets and four signatures a production IDS would need a continuously updated signature database and machine-learning-based anomaly detection to catch novel attacks.
- The design assumes a reasonably mature IT team capable of running a Kerberos KDC and internal CA; a smaller organization might need managed or cloud-based equivalents instead.
- Physical security, insider-threat controls and detailed incident-response procedures were out of scope for this assignment, though they are acknowledged as necessary companions to the technical architecture.

13.3 Possible Improvements

- Add a Security Information and Event Management (SIEM) correlation layer with automated response playbooks, so that detection (IDS/IPS) and action (blocking, isolation) close the loop faster than a human analyst alone could.
- Introduce a zero-trust segmentation model internally, so that Kerberos-based authentication is re-checked per-resource rather than trusted for the duration of a session.
- Move toward post-quantum-resistant key exchange algorithms for the IPsec and TLS layers ahead of long-term regulatory or industry requirements, given how long some enterprise systems stay in service.
- Run a real load test of the proposed mechanisms on representative hardware to replace the indicative performance figures with organization-specific data.

14. References

1. W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed., Pearson, 2020.
2. W. Stallings and L. Brown, *Computer Security: Principles and Practice*, 4th ed., Pearson, 2018.
3. National Institute of Standards and Technology (NIST), FIPS 180-4: Secure Hash Standard (SHS), 2015.
4. National Institute of Standards and Technology (NIST), FIPS 186-5: Digital Signature Standard (DSS), 2023.
5. S. Kent and K. Seo, RFC 4301: Security Architecture for the Internet Protocol, IETF, 2005.
6. E. Rescorla, RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3, IETF, 2018.
7. C. Neuman, T. Yu, S. Hartman and K. Raeburn, RFC 4120: The Kerberos Network Authentication Service (V5), IETF, 2005.
8. D. Cooper et al., RFC 5280: Internet X.509 Public Key Infrastructure Certificate and CRL Profile, IETF, 2008.
9. B. Ramsdell and S. Turner, RFC 5751: S/MIME Version 3.2 Message Specification, IETF, 2010.
10. J. Callas, L. Donnerhake, H. Finney, D. Shaw and R. Thayer, RFC 4880: OpenPGP Message Format, IETF, 2007.
11. Python Software Foundation, Python 3.11 Language Reference, 2023, <https://docs.python.org/3/>.
12. Python Cryptographic Authority, cryptography library documentation, <https://cryptography.io/>.
13. OWASP Foundation, OWASP Top Ten Web Application Security Risks, 2021, <https://owasp.org/www-project-top-ten/>.
14. United Nations, The 17 Sustainable Development Goals, <https://sdgs.un.org/goals>.
15. Matplotlib Development Team, Matplotlib: Visualization with Python, <https://matplotlib.org/>.

Note: No dataset was used in this assignment; all sample packets, messages and traffic values in Sections 6–9 are illustrative test inputs created for demonstration purposes, and the performance figures in Section 10 are indicative values informed by typical published benchmarks rather than a specific cited dataset.